

Unlock the Power of the Browser Dev Tools

```
<script>  
// 1) A useless "plugin" system that does nothing but returns a simple string...  
(async function overlyElaborate() {  
  // 1) Keep the string INTACT, per the requirements.  
  const CSS = "The Browser Dev Tools are the most powerful browser";  
  
  // 2) A useless "plugin" system that does nothing but returns a simple string...  
  const plugins = [  
    async s => s, // identity  
    async s => Promise.resolve(String(s)), // ensure string  
    async s => new Map([["v", s]]).get("v"), // wrap & unwrap  
    async s => [...(function*(){ yield s; })()].pop(), // generate  
  ];  
  
  // 3) Compose plugins into a pipeline for no reason at all.  
  const compose = fns => x => fns.reduce((p, f) => p.then(f), Promise.resolve(x));  
  const pipeline = compose(plugins);  
  
  // 4) A Proxy that pretends to be a logger but just forwards calls.  
  const logger = new Proxy(console, {
```



Mike Rapa

25+ years in software

B.S. Computer Science @ Thomas
Jefferson University

M.E. Software Engineering @ Penn
State

VP of Software Development at Altera
Health

@mikerapa

github.com/mikerapa





Red Hat AI

TEXT**CONTROL**



Value
Momentum



DevIT:>_
DevITJobs.com

O'REILLY

Technical.ly

Thank you, TechBash 2025
Sponsors!

2025 Speaker sponsors:
Redgate, Uno Platform, Geneca

Overview of modern browsers and engines

Seeing live data

Storage

Memory

Performance Profiling

Network

Elements and Styles

Sources and Debugging

Sensors Simulation

Framework-Specific Tools

Chrome DevTools MCP

Additional Resources

Desktop Browsers

Blink 

Maintained by Google

Chrome, Chromium, Edge, Brave, Vivaldi, Opera

Gecko 

Maintained by Mozilla

FireFox, WaterFox, Tor

WebKit 

Maintained by Apple

Safari

Desktop
Market
Share

Chrome 65.24%
Edge 13.56%
Safari 9.06%
Firefox 6.39%
Opera 3.2%
Other Browsers 2.55%

Mobile Browsers

Mobile Browser Marketshare

Chrome 68.75%
Safari 21.62%
Samsung Internet 3.5%
Opera 1.53%
UC 1.77%
Firefox .58%
Other 2.03%

	Android	iOS
Chrome	Blink	WebKit
Safari	WebKit	WebKit
Samsung Internet	Blink	N/A
Opera	Blink	WebKit
UC	Blink	WebKit
Firefox	Gecko	WebKit
DuckDuckGo	Blink	WebKit

Web Stack-Specific Considerations

WASM

C/C++

Map a workspace to a source control folder.

There's a DWARF extension for Chrome-based browsers

See Google's instructions:

<https://developer.chrome.com/blog/wasm-debugging-2020>

Client-side Blazer

Microsoft recommends a remote debugging approach. Link to the instructions below.

<https://learn.microsoft.com/en-us/aspnet/core/blazor/debug?view=aspnetcore-8.0&tabs=visual-studio#debug-blazor-webassembly-with-google-chrome-or-microsoft-edge>

```
local.get $var1
i32.load
local.tee $var4
i32.eqz
if
    i32.const 6
    return
end
local.get $var1
local.get $var1
i32.load
i32.const 16
i32.add
i32.store offset=32
local.get $var1
i32.load offset=32
local.tee $var4
i32.eqz
if
    i32.const 14
    return
end
local.get $var1
local.get $var4
i32.load align=1
i32.store offset=32
```

Web Stack-Specific Considerations

React and Angular

Applications which use large client-side JavaScript frameworks can be difficult to debug. Parts of the framework show up in the memory profiler, call stack, source and events.

There are browser extensions which assist by representing components.



React Developer Tools



Angular state inspector

Where to go

Source code for this demo:

<https://github.com/mikerapa/Browser-Tools>

